Hoot: Progress Report 1

Predictive Household Inventory Tracker

Michael Hannan

Week 9 — October 22, 2026

Where We Are

- This is the Week 9 progress check from the project breakdown.
- Original plan for today: basic database, item/purchase save-and-load, and a simple dashboard.
- What works: a working app demo in the browser first.
- The server and database are started, but not finished yet.
- This talk covers three things: **Done**, **TODO**, and **Update**.

Task Done — What You Can Use Today

- **Dashboard** — items sorted by how soon they may run out, with clear status labels.
- **Restock calendar** — a two-week view of upcoming restock dates.
- **Add item / Log purchase** — simple forms to track staples and record what you bought.
- **Item page** — purchase history, edit or delete, and a chart of the predicted restock.
- **One-time list** — a checklist for one-off needs that should not be predicted.

Task Done — How Predictions Work (For Now)

- New items use a manual guess: "I usually buy this every X days."
- After three purchases, Hoot learns the gap between buys and estimates the next date.
- Demo data (milk, eggs, paper towels, and more) is saved in the browser.
- The UI is set up so it can later talk to a real server without a full redesign.
- Planning and design docs are written: breakdown, project plan, product notes, and UI flows.

Task Done — By Project Area

Area	Status
Frontend	Mostly done: dashboard, forms, calendar, item page, one-time list
Prediction	Works in the browser demo; not yet saved on the server
Backend / API	Project started; real item and purchase endpoints still needed
Database	Tables are planned; not built in PostgreSQL yet
Barcode lookup	Placeholder on the item page; no live product lookup yet
Testing & Deploy	Manual demo testing only; no packaging or deploy setup yet

Task TODO — Next (Weeks 10–12)

- Build the database tables for users, items, purchases, and predictions.
- Finish the server API so items and purchases can be saved for real.
- Connect the front end to that API instead of browser-only storage.
- Move prediction math onto the server and keep a history of each estimate.
- Add barcode lookup so scanning can fill in product details.

Task TODO — Later

- Login and register screens.
- Check how accurate predictions were over time.
- More testing, bug fixes, and optional Docker deploy.
- Nice-to-haves later: shared households, shopping mode, price trends, reminders.
- Final week: demo slides, demo video, final report, and code hand-in.

Task Update — Plan vs. Reality

- **Ahead on the UI:** we planned a basic dashboard; we have ranking, calendar, charts, and full add/edit/delete flows.
- **Behind on the database:** save-and-load on the server was due now; that work comes next.
- **Prediction early, but local:** the learning math works in the demo; it still needs to live on the server.
- **One-time list early:** that checklist was planned later and is already in the demo.

Task Update — What Comes Next

- First job: finish the database and server so the app does not rely on browser storage.
- Next: barcode and product lookup once the core tables exist.
- Goal for Progress Report 2 (Nov 10): live API, server predictions, and barcode support started.
- Watch-out: keep the same prediction rules when moving from demo to server.